CRYSTALS-Kyber (Kyber-512) PKE Implementation in C Using NTT

Technical Report, Mathematical Derivations, Interactive Walkthroughs, and Live Demonstration Guide

Companion Engineering Documentation for `Kyber_c.c`

Abstract

This report provides a complete mathematical and systems-level analysis of the standalone C implementation of **Kyber-512 Public-Key Encryption (IND-CPA PKE)** in `Kyber_c.c`. Designed for code presentations and technical defense, it bridges lattice-based cryptography with low-level C optimizations: ring algebra over $R_q = \mathbb{Z}_{3329}[X]/(X^{256}+1)$, division-free 16-bit Montgomery (62209) and Barrett (20159) reductions, the 7-layer Number Theoretic Transform (NTT) and its inverse butterfly sign cancellation in `invntt()`, Montgomery domain invariants across key generation, encryption, and decryption, a concrete numerical trace of a bit traveling through the code, and live code experiments.

1 Executive Overview and Codebase Architecture

Classical public-key cryptosystems (RSA, ECC) are vulnerable to polynomial-time quantum attacks via Shor's algorithm. **CRYSTALS-Kyber** (standardized by NIST as **ML-KEM** in FIPS 203) is based on the hardness of the **Module Learning With Errors (M-LWE)** problem over module lattices. At the core of the IND-CCA2 KEM sits the **IND-CPA Public-Key Encryption (PKE)** scheme implemented from scratch in `Kyber_c.c` using fixed-width 16-bit and 32-bit integer arithmetic. Table 1 maps every C function in `Kyber_c.c` to its mathematical role.

Table 1: Structural Map of `Kyber_c.c` Functions and Primitives

C Function / Symbol	Lines	Mathematical & Systems Role
`KYBER_N`, `KYBER_Q`, `KYBER_K`	13–15	Ring degree $n = 256$, prime modulus $q = 3329$, module rank $k = 2$ (Kyber-512).
`bitrev7`, `rotl`, `prng32`, `prng_bit`	21–53	7-bit index reversal and Xoshiro128** pseudo-random bit-pool generator.
`init_zetas`	56–71	Precomputes 128 bit-reversed powers of $\zeta = 17$ scaled into the Montgomery domain ($R = 2^{16}$).
`montgomery_reduce`, `fqmul`	74–95	Division-free reduction computing $a \cdot 2^{-16} \bmod 3329$ and Montgomery multiplication.
`barrett_reduce`	82–90	Division-free centered reduction computing $a \bmod 3329$ via fixed-point reciprocal $v = 20159$.
`ntt`, `invntt`	98–135	In-place 7-layer Cooley-Tukey forward NTT and Gentleman-Sande inverse NTT.
`basemul`, `poly_basemul_montgomery`	138–151	Degree-1 polynomial multiplication modulo $(X^2 - \zeta^{2\text{br}_7(i)+1})$ across 128 residue pairs.
`poly_add`, `poly_sub`, `poly_reduce`, `poly_tomont`	154–177	Coefficient-wise vector algebra and domain conversion ($\times R^2 \bmod q$, where $R^2 \equiv 1353$).
`poly_getnoise_eta3`, `poly_getnoise_eta2`	179–201	Centered Binomial Distribution (CBD_η) samplers generating small errors in $[-\eta, \eta]$.
`compress_poly`, `decompress_poly`	204–220	Lossy quantization $\mathbb{Z}_q \leftrightarrow \mathbb{Z}_{2^d}$ ($d_u = 10$ bits for **u**, $d_v = 4$ bits for v).
`poly_frommsg`, `poly_tomsg`	222–241	Maps 256-bit messages to $\{0, 1665\}^n$ and strips LWE noise during decoding.
`Kyber_PKE_KeyGen`, `Encrypt`, `Decrypt`	249–359	Core IND-CPA PKE key generation, encryption, and decryption routines.
`get_cycles`, `main`	362–454	Hardware timestamp counter (`__rdtsc`) benchmarking (1000 runs) and verification suite.

2 Mathematical Foundations, PRNG, and Noise Sampling

2.1 The Polynomial Ring R_q and Parameter Rationale

Kyber operates over the cyclotomic polynomial ring $R_q = \mathbb{Z}_q[X]/(X^n+1)$ with $n = 256$ (`KYBER_N`) and prime $q = 3329$ (`KYBER_Q`). Every polynomial $f(X) = \sum_{i=0}^{255} f_i X^i \in R_q$ is stored as an array `int16_t r[256]`. Multiplication modulo $X^{256} + 1$ is a negacyclic convolution ($X^{256} \equiv -1$). The parameters are chosen for three engineering reasons:

1. **16-bit Signed Headroom ($q = 3329 < 2^{12}$):** Coefficients occupy 12 bits (or $[-1664, 1664]$ centered), leaving 4 bits of overflow headroom inside `int16_t` so additions (`poly_add`) and subtractions (`poly_sub`) require no immediate modular reduction.
2. **Incomplete NTT Prime ($q = 13 \times 2^8 + 1$):** Because $256 \mid (q-1)$ but $512 \nmid (q-1)$, primitive 256-th roots of unity exist modulo 3329 ($\zeta = 17$), factoring $X^{256} + 1$ into 128 quadratic trinomials $X^2 - \zeta^{2\text{br}_7(i)+1}$.
3. **Module Rank ($k = 2$):** Instead of increasing ring degree n, Kyber uses $k \times k$ matrices over R_q. Setting `KYBER_K = 2` yields a lattice dimension of $k \cdot n = 512$ (NIST Security Level 1, equivalent to AES-128).

Module-LWE Hardness: Given a uniform matrix $\mathbf{A} \in R_q^{k \times k}$, a small secret vector $\mathbf{s} \in R_q^k$, and a small error vector $\mathbf{e} \in R_q^k$, the noisy vector $\mathbf{t} = \mathbf{A}\mathbf{s} + \mathbf{e} \pmod q$ is computationally indistinguishable from uniform random. Without **e**, **s** could be found via Gaussian elimination; with **e**, recovering **s** requires solving the Shortest Vector Problem (SVP) in a 512-dimensional lattice.

2.2 Bit-Pool PRNG, Rejection Sampling, and Centered Binomial Distribution

To remain dependency-free, `Kyber_c.c` implements a 32-bit **Xoshiro128**** generator (`prng32()`, lines 29–40). To avoid wasting 31 random bits when a single coin toss is needed, `prng_bit()` (lines 42–53) caches one 32-bit word in a `static uint32_t bits` register and shifts out one bit per call, refilling only once every 32 calls.

- **Rejection Sampling for $\hat{\mathbf{A}}$ (lines 253–258):** Reducing random integers via `prng32() % 3329` introduces statistical bias because $3329 \nmid 2^{32}$. Instead, the code masks to 13 bits (`val = prng32() & 0x1FFF` in $[0, 8191]$) and repeats `while(val >= 3329)`, guaranteeing an exact uniform distribution over $[0, 3328]$. Furthermore, `pk_A` is assumed to be sampled *directly in the NTT domain*, saving $k^2 = 4$ forward NTT transforms.
- **Centered Binomial Distribution (CBD$_\eta$, lines 179–201):** Secret and error coefficients are sampled by summing η random bits a_j and subtracting η random bits b_j: $x = \sum_{j=1}^{\eta}(a_j - b_j) \in [-\eta, \eta]$. This runs in constant time without floating-point operations. For $\eta_1 = 3$ (`poly_getnoise_eta3`), values span $[-3, 3]$ with probabilities $\{1, 6, 15, 20, 15, 6, 1\}/64$ and variance $\eta_1/2 = 1.5$. For $\eta_2 = 2$ (`poly_getnoise_eta2`), values span $[-2, 2]$ with probabilities $\{1, 4, 6, 4, 1\}/16$ and variance $\eta_2/2 = 1.0$.

3 Constant-Time Modular Arithmetic: Montgomery and Barrett

Hardware division (`/`) and modulo (`%`) instructions take 10–40+ variable clock cycles on x86 CPUs, leaking secret keys via timing side-channels. All runtime polynomial arithmetic in `Kyber_c.c` replaces division with constant-time bitwise shifts using **Montgomery** and **Barrett** reductions.

3.1 Signed 16-Bit Montgomery Reduction (`montgomery_reduce` and `fqmul`)

Montgomery reduction replaces division by $q = 3329$ with division by $R = 2^{16} = 65536$ (an exact right shift `>> 16`). Given a 32-bit product a ($|a| < 2^{15}q$), we subtract a multiple $t = u \cdot q$ whose lower 16 bits match a identically. Using the modular inverse q^{-1} mod 2^{16}:

$$3329 \times 62209 = 207,093,761 = 3160 \times 2^{16} + 1 \equiv 1 \pmod{2^{16}} \tag{1}$$

Lines 74–80 implement this in four arithmetic operations:

```
int16_t montgomery_reduce(int32_t a) {
    int16_t u = (int16_t)(a * 62209);  // u = (a * q^-1) mod 2^16 (signed 16-bit truncation)
    int32_t t = (int32_t)u * KYBER_Q;  // t = u * 3329 (matches lower 16 bits of a)
    t = a - t;                         // Lower 16 bits of (a - t) are guaranteed 0x0000
    t >>= 16;                          // Exact division by R = 2^16
    return (int16_t)t;                 // Returns a * R^-1 mod 3329 in (-3329, 3329)
}
```

Because $t = u \cdot q$ is a multiple of q, $a - t \equiv a \pmod q$. Modulo 2^{16}, $t \equiv (a \cdot q^{-1})q \equiv a \pmod{2^{16}}$, so $a - t$ is an exact multiple of 2^{16}. Thus, `fqmul(a, b)` (line 92) computes $a \cdot b \cdot 2^{-16} \bmod 3329$. To compensate for the 2^{-16} factor, constants in `zetas[128]` are pre-scaled by $R = 2^{16} \equiv 2285 \pmod{3329}$.

3.2 Centered Barrett Reduction (`barrett_reduce`)

After polynomial additions, coefficients must be reduced modulo 3329 without multiplying by R^{-1}. Barrett reduction approximates $1/q$ using the fixed-point reciprocal $v = \lfloor (2^{26} + q/2)/q \rfloor = \lfloor 67,110,528/3329 \rfloor = 20159$

(lines 82–90):

$$t = \left\lfloor \frac{20159 \cdot a + 2^{25}}{2^{26}} \right\rfloor = \left\lfloor \frac{a}{3329} \right\rceil, \qquad \texttt{barrett_reduce}(a) = a - 3329 \cdot t \in [-1664, 1664] \tag{2}$$

Adding 2^{25} (`1 << 25`) before shifting right by 26 bits implements nearest-integer rounding, returning the centered remainder around zero.

4 The Number Theoretic Transform (NTT) and Base Multiplication

The NTT reduces polynomial multiplication from $\mathcal{O}(n^2)$ ($65,536$ multiplications) to $\mathcal{O}(n \log n)$ butterfly operations plus 128 degree-1 base multiplications. Using the primitive 256-th root of unity $\zeta = 17$ ($17^{128} \equiv -1 \pmod{3329}$), the Chinese Remainder Theorem splits $X^{256} + 1 = X^{256} - \zeta^{128}$ across 7 binary tree layers ($\texttt{len} = 128, 64, 32, 16, 8, 4, 2$) into 128 quadratic factors:

$$R_q \cong \prod_{i=0}^{127} \mathbb{Z}_{3329}[X] / \left(X^2 - \zeta^{2\mathsf{br}_7(i)+1}\right) \tag{3}$$

In `init_zetas()` (lines 56–71), the code precomputes $\texttt{zetas}[i] = \text{center}(17^{\mathsf{br}_7(i)} \cdot 2285 \bmod 3329)$, where $\mathsf{br}_7(i)$ is the 7-bit reversal (`bitrev7(i)`) and $2285 \equiv 2^{16} \pmod{3329}$ converts each twiddle factor into the Montgomery domain.

4.1 Forward Butterfly (`ntt`) and the `invntt` Negative-Root Sign Cancellation

In `ntt()` (lines 98–112), index `k` increments from 1 to 127, applying the Cooley-Tukey butterfly $t \leftarrow \texttt{fqmul}(\zeta, r[j + \text{len}])$, $r[j + \text{len}] \leftarrow r[j] - t$, $r[j] \leftarrow r[j] + t$.

In `invntt()` (lines 114–135), the code traverses `zetas[k--]` backward from $k = 127$ down to 1 without storing a separate inverse table. Why does line 128 compute `r[j + len] = r[j + len] - t` instead of `t - r[j + len]`?

1. A forward NTT block uses ζ^m ($m \in [1, 127]$), while the reverse traversal in `invntt` fetches ζ^{128-m}.
2. Since $\zeta^{128} \equiv -1 \pmod{3329}$, we have $\zeta^{128-m} = \zeta^{128} \cdot \zeta^{-m} \equiv -\zeta^{-m} \pmod{3329}$ (the **negative** modular inverse).
3. Reversing the subtraction order on line 128 negates the difference so the two minus signs cancel algebraically:

$$\big(r[j + \text{len}] - r[j]\big) \cdot \big(-\zeta^{-m}\big) = +\big(r[j] - r[j + \text{len}]\big) \cdot \zeta^{-m} \tag{4}$$

Finally, lines 132–134 multiply every coefficient by $f = 1441$ via `fqmul(r[j], 1441)`. Since $1441 \equiv \frac{R^2}{128} = \frac{2^{32}}{2^7} \pmod{3329}$, applying `fqmul` multiplies by $\frac{R^2}{128} \cdot R^{-1} = \frac{R}{128} \bmod 3329$. This simultaneously divides by 128 (completing the 2^{-7} inverse NTT scaling) and multiplies by R^1, which cancels the R^{-1} factor left behind by `poly_basemul_montgomery`!

4.2 Base Multiplication (`basemul`) and Montgomery Domain Tracking

In the NTT domain, multiplying two residue pairs $(a_0 + a_1 X)(b_0 + b_1 X) \bmod (X^2 - \gamma)$ where $\gamma = \pm\texttt{zetas}[64 + i]$ yields:

$$r[0] = a_0 b_0 + a_1 b_1 \gamma, \qquad r[1] = a_0 b_1 + a_1 b_0 \tag{5}$$

implemented in 5 calls to `fqmul` in `basemul()` (lines 138–144). Because `fqmul` multiplies two unscaled operands by R^{-1}, the output of `poly_basemul_montgomery` carries a factor of R^{-1}. Table 2 explains why `poly_tomont()` is required in `KeyGen` but never in `Encrypt` or `Decrypt`.

5 Compression, Message Encoding, and Interactive Numerical Trace

5.1 Message Encoding, Lossy Quantization, and Decryption Error Bound

- **Message Mapping (`poly_frommsg` & `poly_tomsg`, lines 222–241):** To protect 1-bit message values from additive LWE noise, `poly_frommsg` scales each bit $b \in \{0, 1\}$ to $b \cdot \lceil q/2 \rceil \in \{0, 1665\}$, placing 0 and 1 at

Table 2: Montgomery Factor ($R = 2^{16} \bmod 3329$) Audit Across the Protocol

Phase	Code Operation	Scale	Mathematical Explanation
KeyGen	`ntt(sk_s)`, `ntt(e)`	R^0	`zetas` are R^1 and `fqmul` applies R^{-1}, preserving scale R^0.
KeyGen	`poly_basemul_montgomery(pk_t, pk_A, sk_s)`	R^{-1}	Multiplying two R^0 polynomials via `fqmul` yields $\hat{\mathbf{A}} \circ \hat{\mathbf{s}} \cdot R^{-1}$.
KeyGen	`poly_tomont(pk_t)` ($\times 1353 \cdot R^{-1}$)	R^0	Multiplies by $1353 \equiv R^2 \bmod q$ and reduces by R^{-1}, restoring $\hat{\mathbf{t}}$ to R^0 before adding $\hat{\mathbf{e}}$.
Encrypt	`poly_basemul_montgomery` → `invntt(u)`, `invntt(v)`	R^0	`basemul` introduces R^{-1}; `invntt`'s final $\times 1441$ applies $\frac{R}{128}$, canceling R^{-1} automatically!
Decrypt	`poly_basemul_montgomery` → `invntt(w)`	R^0	Same cancellation: $\hat{\mathbf{s}}^T \circ \hat{\mathbf{u}}'$ gets R^{-1} from `basemul` and R^1 from `invntt`, yielding w at R^0.

opposite poles of $\mathbb{Z}_{3329}$. During decryption, `poly_tomsg` computes $\lfloor (2x + 1664)/3329 \rfloor \ \& \ 1$, which decodes to 1 whenever $x \in [833, 2496]$ and 0 otherwise. Thus, **any total noise magnitude strictly less than $\lfloor q/4 \rfloor = 832$ decodes with zero error**.

- **Ciphertext Compression (`compress_poly` & `decompress_poly`, lines 204–220):** Quantizes $\mathbb{Z}_q \to \mathbb{Z}_{2^d}$ via $\lfloor (2^d x + 1664)/3329 \rfloor$ mod 2^d and expands back via $\lfloor (3329y + 2^{d-1})/2^d \rfloor$. Compressing $\mathbf{u}$ to $d_u = 10$ bits (640 bytes) adds at most ± 2 rounding error; compressing v to $d_v = 4$ bits (128 bytes) adds at most ± 104 error, reducing total ciphertext size by 33.3% (from $1,152$ bytes to 768 bytes).
- **Algebraic Mask Cancellation (`Kyber_PKE_Decrypt`, lines 336–359):** Subtracting $w = \mathbf{s}^T \mathbf{u}'$ from v' yields:

$$v' - \mathbf{s}^T \mathbf{u}' = \left(\mathbf{s}^T \mathbf{A}^T \mathbf{r} + \mathbf{e}^T \mathbf{r} + e_2 + 1665m + c_v\right) - \left(\mathbf{s}^T \mathbf{A}^T \mathbf{r} + \mathbf{s}^T (\mathbf{e}_1 + \mathbf{c}_u)\right) = 1665m + \underbrace{\left(\mathbf{e}^T \mathbf{r} - \mathbf{s}^T (\mathbf{e}_1 + \mathbf{c}_u) + e_2 + c_v\right)}_{\text{Total Noise } |E_\infty| \ll 832} \tag{6}$$

5.2 Interactive Numerical Walkthrough: Tracing Index 0 Through the C Code

Let us trace concrete integer values through the exact C expressions in `Kyber_c.c`:

Part A: Tracing `fqmul(-758, 500)` and `barrett_reduce(4120)`

1. In `ntt()`, multiplying `zeta` $= -758$ ($17^{64} \cdot 2^{16} \bmod 3329$) by 500 gives $a = -379,000$. Inside `montgomery_reduce`: `u = (int16_t)(-379000 * 62209)` $= 8904$; `t = 8904 * 3329` $= 29,641,416$; `a - t` $= -30,020,416$; shifting right by 16 (`t >>= 16`) yields exactly -458 (with zero remainder, matching $17^{64} \times 500 \equiv 2871 \equiv -458 \pmod{3329}$).
2. For $a = 4120$ in `barrett_reduce`: `shift_val` $= 20159 \times 4120 + 2^{25} = 116,609,512$; `t = shift_val >> 26` $= 1$; returning $4120 - 1 \times 3329 = 791$.

Part B: Tracing Plaintext Bit $b = 1$ from Encryption to Decryption at Coefficient Index 0

Stage 1: Encode (`poly_frommsg`): Plaintext bit $b = 1$ is mapped to `m_poly[0]` $= (3329 + 1)/2 = 1665$.

Stage 2: Encrypt (`Kyber_PKE_Encrypt`): Suppose at index 0, the inner product $(\mathbf{As})^T \mathbf{r} \equiv 2100$ and LWE noise $\mathbf{e}^T \mathbf{r} + e_2 = +45$. Adding `m_poly[0]` $= 1665$ gives $v[0] = 2100 + 45 + 1665 = 3810 \xrightarrow{\texttt{barrett_reduce}} 481$.

Stage 3: Compress (`compress_poly`, $d = 4$): `v_comp[0]` $= \lfloor ((481 \ll 4) + 1664)/3329 \rfloor = \lfloor 9360/3329 \rfloor = 2$ (stored as 4-bit 0010_2).

Stage 4: Decompress (`decompress_poly`, $d = 4$): Receiver unpacks `v_comp[0]` $= 2 \implies v'[0] = (2 \times 3329 + 8) \gg 4 = 416$ (quantization shift of -65).

Stage 5: Unmask (`Kyber_PKE_Decrypt`): Receiver computes $w[0] = (\mathbf{s}^T \mathbf{u}')[0] = 2100 - 18 = 2082$. Subtracting gives `m_out_poly[0]` $= 416 - 2082 = -1666 \xrightarrow{\texttt{barrett_reduce}} 1663$ (net noise is just -2 from 1665!).

Stage 6: Decode (`poly_tomsg`): $t = \lfloor ((1663 \ll 1) + 1664)/3329 \rfloor \ \& \ 1 = \lfloor 4990/3329 \rfloor \ \& \ 1 = 1$. Exact recovery!

6 Live Code Experiments, Benchmark Analysis, and Presentation Q&A

6.1 Four Interactive Code Experiments for Live Demonstrations

Exp 1: Prove the `invntt` Negative-Root Fix (Line 128): Change `r[j + len] = r[j + len] - t;` to `t - r[j + len];`. The program compiles cleanly but outputs `[-] ERROR! Decryption failed`, proving visually that reverse traversal of `zetas` supplies $-\zeta^{-1}$ and requires reversed subtraction to cancel the minus sign.

Exp 2: Break the Montgomery Domain Invariant (Line 281): Comment out `poly_tomont(pk_t[i]);` in `KeyGen`. Decryption fails immediately because $\hat{\mathbf{t}}$ retains an uncompensated R^{-1} factor from `basemul`, preventing $\mathbf{s}^T \mathbf{u}$ from canceling $\mathbf{t}^T \mathbf{r}$.

Exp 3: Probe the Compression Noise Threshold (Lines 333 & 344): Lower d_v in `compress_poly`/`decompress_poly` from 4 to 2 (still passes most runs) and then 1 (fails due to ± 832 quantization error exceeding the $\lfloor q/4 \rfloor = 832$ noise ceiling).

Exp 4: Scale Security Levels (Line 15): Change `#define KYBER_K 2` to 3 (Kyber-768) or 4 (Kyber-1024). Observe that `NTT` and `InvNTT` cycle counts stay constant while `KeyGen` and `Encrypt` scale with matrix dimension k^2.

6.2 Hardware Cycle Benchmark Breakdown (`__rdtsc`)

Table 3: Algorithmic Complexity and Bottleneck Analysis in `Kyber_c.c` ($k = 2$)

Function	NTT	InvNTT	Basemul	CBD	Why It Exhibits Its Measured Cycle Count
`ntt(r)`	1	0	0	0	896 butterflies; lazy addition requires zero intermediate Barrett reductions.
`invntt(r)`	0	1	0	0	Slightly slower than `ntt`: executes 896 `barrett_reduce` calls + 256 final `fqmul` calls.
`Kyber_PKE_KeyGen`	4	0	4	4	Dominated by rejection sampling 1,024 uniform entries for $\hat{\mathbf{A}}$ + 4 CBD polynomials.
`Kyber_PKE_Encrypt`	2	3	6	5	Samples 5 noise polynomials, runs 5 transforms, 6 base multiplications, and compression.
`Kyber_PKE_Decrypt`	2	1	2	0	**Fastest PKE phase:** zero PRNG/CBD calls, zero matrix multiplications (only inner product $\hat{\mathbf{s}}^T \circ \hat{\mathbf{u}}$).

6.3 Anticipated Presentation Q&A Reference

- **Q: How does this IND-CPA PKE become the full NIST ML-KEM (IND-CCA2)?** By applying the **Fujisaki-Okamoto (FO) transform**: during decapsulation, the receiver decrypts m', re-runs deterministic `Kyber_PKE_Encrypt` using coins hashed from (m', pk), and checks in constant time that the re-encrypted ciphertext matches the received ciphertext.
- **Q: Why is `sk_s` stored permanently in the NTT domain?** `KeyGen` transforms `sk_s` via `ntt(sk_s[i])` on line 271. Keeping it in the NTT domain saves $k = 2$ forward NTT calls during every `Kyber_PKE_Decrypt` execution.
- **Q: Compilation Command:** `gcc -O3 -march=native -Wall -Wextra Kyber_c.c -o kyber512_bench && ./kyber512_bench`